KB FRONTEND

Table Architecture and Behavior Reference

A detailed guide to table extensions, commands, toolbar state, headers, borders, resizing, dragging, selection, and production hardening

Repository	C:\gamalearn\kb-frontend
Primary stack	Next.js 16.2.7, React 19.2.4, Tiptap 3.26.0, ProseMirror tables
Document date	June 10, 2026
Scope	All table-related extensions, commands, plugins, toolbar UI, utilities, styles, and tests
Verification	40 Vitest tests, 10 Playwright tests, lint, TypeScript, browser inspection, production build

Design intent

The implementation treats table state as editor data, DOM geometry as a temporary interaction surface, and every command boundary as potentially invalid.

Contents

Reading guide: Sections 1-11 explain the system by behavior. Section 12 is the detailed file map. Sections 13-17 focus on edge cases, hardening, verification, and future maintenance.

1. Executive Architecture Overview

The table implementation is a layered Tiptap/ProseMirror subsystem. The editor schema stores durable table data; commands make validated document mutations; custom plugins manage geometry-heavy mouse interactions; React toolbar components expose the supported operations; and CSS renders persistent attributes and temporary interaction handles.

> **Core invariant:** Persistent state belongs on ProseMirror nodes. Temporary preview state belongs in the DOM or plugin-local session state. A preview is restored before the final node mutation is committed.

Architecture at a glance

```
EditorWorkspace / KnowledgeBaseEditor
        |
        v
EditorToolbar + TableControls -----> safe command boundary
        |                                  |
        v                                  v
Tiptap table extensions ----------> ProseMirror transactions
        |                                  |
        v                                  v
Custom table node attributes       document history / undo-redo
        |
        +--> TableView applies width, offset, and borders to DOM
        +--> Plugins preview and commit row resize, outer resize, and drag
        +--> CSS renders cells, borders, selections, and handles
```

Design goals

- Keep table behavior modular and colocated under src/features/editor/table.
- Preserve semantic header intent after structural changes, including first-row and first-column mutations.
- Return false instead of throwing when editor, selection, table map, DOM, or plugin state is invalid.
- Make geometry changes undoable as single history actions.
- Keep toolbar enabled/disabled state consistent with the action that will actually run.
- Persist border, width, offset, and row-height choices through JSON and HTML serialization.

2. Runtime Architecture and Data Flow

2.1 Editor startup

18. KnowledgeBaseEditor creates a client-side Tiptap Editor with getEditorExtensions().
19. getEditorExtensions() registers the standard editor extensions, TableKit with selected defaults disabled, and the custom tableExtensions array.

20. The custom KnowledgeBaseTable replaces the default table node, while custom cell and header extensions add persisted rowHeight attributes.
21. RowResizePlugin, TableOuterResizePlugin, and TableDragHandlePlugin are wrapped as Tiptap extensions and installed as ProseMirror plugins.
22. EditorToolbar renders the global table creation picker and the context-sensitive TableControls component.

2.2 Mutation lifecycle

```
User action
  -> toolbar / keyboard / plugin event
  -> validate editor + active table + selection
  -> create or run ProseMirror transaction
  -> normalize headers or attributes when required
  -> dispatch transaction
  -> history captures one undoable action
  -> TableView and React toolbar re-read the new editor state
```

2.3 Persistent versus temporary state

State category	Stored in	Examples	Why
Persistent document state	ProseMirror node attributes	tableWidthPct, tableOffsetPct, border flags, rowHeight	Serializes to JSON/HTML and participates in undo/redo.
Plugin state	PluginKey state	Active row resize target, active outer resize table	Tracks interaction targets and maps positions through transactions.
Drag-session state	Plugin closure / MouseDragSession	Start coordinates, preview values, dominant drag axis	Exists only for the current pointer interaction.
DOM preview state	Inline styles, data attributes, temporary style element	Live row height, live outer width, live horizontal offset	Gives immediate feedback without producing many history entries.
React toolbar state	useEditorState selector result	Visibility, header flags, border flags, command capabilities	Keeps controls synchronized with current selection and editor state.

3. Extension Configuration and Persisted Schema

3.1 TableKit replacement strategy

The editor still uses Tiptap's TableKit to provide the table row extension and common table infrastructure, but it explicitly disables the default table, tableCell, and tableHeader nodes. Those three nodes are replaced by custom extensions so the application can own attributes, rendering, keyboard behavior, and plugin configuration.

```
TableKit.configure({ table: false, tableCell: false, tableHeader: false })
...tableExtensions
```

3.2 Table extension options

Option	Configured value	Effect
resizable	true	Enables Tiptap's internal column-resizing plugin and TableView.
cellMinWidth	40	Sets the minimum internal column/cell width to 40 pixels.
lastColumnResizable	false	Prevents the final internal column edge from acting as the outer-table resize edge.
draggable	true	Marks the table node as draggable; the custom drag handle controls the interaction.
View	KnowledgeBaseTable View	Applies stored width, offset, and border attributes whenever the table node view is created or updated.

3.3 Persisted table attributes

Attribute	Default	Rendered form	Meaning / constraints
tableWidthPct	100	data-table-width-pct and width/CSS variable	Outer table width, clamped to 10-100 percent.
tableOffsetPct	0	data-table-offset-pct and margin-left/CSS variable	Horizontal offset, clamped so width + offset never exceeds 100 percent.
borderTopEnabled	true	data-table-border-top	Controls the visible top edge.
borderRightEnabled	true	data-table-border-right	Controls the visible right edge.
borderBottomEnabled	true	data-table-border-bottom	Controls the visible bottom edge.
borderLeftEnabled	true	data-table-border-left	Controls the visible left edge.
borderInnerEnabled	true	data-table-border-inner	Controls internal grid lines while allowing outer edges to remain visible.

3.4 Persisted cell attributes

Both tableCell and tableHeader nodes receive the same rowHeight attribute. The value is normalized to a positive integer of at least 20 pixels and rendered as data-row-height plus an inline height style. Storing the value on every unique cell that participates in the row allows row height to survive serialization and complex table structures.

Compatibility behavior: Tables loaded without the custom border attributes default to all borders enabled. Existing HTML width and margin-left percentages are accepted as fallback inputs when data attributes are missing.

4. Command Architecture and Safe Failure Model

4.1 Why commands are wrapped

Direct Tiptap/ProseMirror table commands assume a valid table selection and structurally valid table map. In a UI, those assumptions can be invalidated by asynchronous React updates, read-only mode changes, destroyed editors, malformed imported content, stale positions, or selections that cover an entire table. The command module is the safety boundary that converts those cases into a false result instead of a runtime exception.

4.2 Safe command families

Function	Purpose	Safety behavior
runTableStructureCommand	Insert/delete rows or columns while preserving header intent.	Requires usable active table; catches failures; normalizes headers in the same chain.
canRunTableCommand	Compute accurate toolbar capability state.	Returns false for invalid editor/table state and explicitly blocks full-table row/column deletion.
runTableActionCommand	Run merge, split, delete-table, or header toggle actions.	Requires editable active table and catches command failures.
updateTableBorders	Persist partial border settings.	Only updates an active editable table.
insertTable	Insert a new table with a header row.	Validates editor state and positive integer dimensions.
reorderTableRow	Move a row while preserving edge header semantics.	Validates indexes and performs full header normalization after moveTableRow.

4.3 Validation sequence

23. Reject null or undefined editors.
24. Reject destroyed or read-only editors.
25. Resolve the active table from a table NodeSelection or by walking up from the current selection.
26. Safely construct a TableMap when the operation requires table geometry.
27. Validate command-specific inputs, such as dimensions, row indexes, or selected rectangle coverage.
28. Run the underlying command inside a try/catch boundary.
29. Return false without mutating the document if any prerequisite fails.

Important distinction: canRunTableCommand does more than call editor.can(). ProseMirror can report deletion as available for a selection covering all rows or all columns even though the actual command refuses it. The wrapper checks selectedRect explicitly so toolbar state matches runtime behavior.

5. Header Row and Header Column Behavior

5.1 Header intent

Header state is treated as an edge-level semantic intent: the first row may be a header row, the first column may be a header column, or both may be active. The implementation detects that intent before structural mutation using rowIsHeader and columnIsHeader.

5.2 Why normalization is necessary

Tiptap's structural commands copy neighboring cell types when adding or deleting rows and columns. That behavior can move or duplicate tableHeader node types in ways that do not match the application's edge-header model. For example, inserting a row above the first row may leave the former header row as headers, and deleting the first header column may fail to promote the new first column.

5.3 Normalization algorithm

30. Snapshot hasHeaderRow and hasHeaderColumn before the structural command runs.
31. Run the Tiptap add/delete command in a chain.
32. Resolve the active table from the transaction's updated document.
33. Use TableMap to visit cells along the first two row/column edges for normal structural changes.
34. Set each visited unique cell to tableHeader only when it belongs to an intended header edge; otherwise set it to tableCell.
35. For explicit row reordering, normalize the entire table so a moved former header row is demoted everywhere.
36. Commit normalization in the same transaction/history action as the structure change.

Operation	Expected header result
Insert row above the header row	New first row becomes the header row; displaced former header row becomes normal cells except for an active header column.
Insert row below the header row	Inserted row is normal except for an active first-column header cell.
Delete the header row	The next row is promoted to the header row.
Insert column before header column	New first column becomes the header column; displaced former header column is demoted.
Delete header column	The new first column is promoted to header cells.
Reorder first row away from edge	The new first row is promoted; moved row is demoted except for header-column cells.
Undo / redo	Header semantics return with the same structural transaction.

6. Selection, Deletion, Merge, and Split Behavior

6.1 Active table resolution

An active table is resolved in two ways. If the selection is a NodeSelection of a table, the selected table position is used directly. Otherwise, the implementation walks from selection.$from toward the document root until it finds a node named table. This supports text selections, cell selections, and whole-table selections.

6.2 Selected-cell deletion

The custom table extension overrides Backspace, Mod-Backspace, Delete, and Mod-Delete. When a CellSelection is active, deleteCellSelection clears the contents of selected cells but preserves the table grid. This intentionally differs from Tiptap's default shortcut that deletes the entire table when every cell is selected.

Deletion policy: Keyboard deletion clears cell content. Structural row/column deletion and whole-table deletion are explicit toolbar actions. This reduces accidental destructive behavior.

6.3 Merge and split

Merge and split use Tiptap's table commands through runTableActionCommand. Capability state is computed with canRunTableCommand, so Merge is enabled only for a valid rectangular multi-cell CellSelection and Split is enabled only for a merged cell. Integration tests verify that TableMap remains structurally valid after both actions.

7. Toolbar Architecture and State Synchronization

7.1 Global creation control

TableCreationPicker is always available in the main EditorToolbar while the editor is editable. It offers a 6-by-8 visual grid plus manual dimensions. Manual values are parsed, clamped to 1-100 rows and 1-20 columns, and passed through the safe insertTable command.

7.2 Contextual table controls

TableControls appears only when an editable editor has an active table. Its state is derived with useEditorState, which causes React to recompute the selector whenever relevant editor state changes. This keeps toolbar visibility, active header flags, border checkboxes, and enabled/disabled command states synchronized with selection changes and document mutations.

Toolbar group	Controls	Command path
Insert	Row above/below, column before/after	runTableStructureCommand

Toolbar group	Controls	Command path
Cells	Merge, split	runTableActionCommand
Delete	Delete row, column, or table	Structure wrapper for row/column; action wrapper for table
Headers	Toggle header row or column	runTableActionCommand
Borders	All, outer, inner, and individual edges	updateTableBorders

7.3 Inactive state

The inactive state is a complete typed object with visibility false, disabled capabilities, no header intent, and default border values. Returning a stable inactive shape makes the selector predictable and prevents conditional property access from leaking into the component.

8. Resizing and Dimensions

8.1 Internal column resizing

Internal column resizing is supplied by Tiptap's resizable table extension. It updates column widths inside the table but must not change the custom outer table width. Setting lastColumnResizable to false reserves the table's rightmost boundary for the outer resize plugin.

8.2 Outer table resizing

37. Only an active editable table can expose the outer resize handle.
38. Hit detection requires the pointer to be within 8 pixels of the right edge and vertically inside the table rectangle.
39. The drag session records startX, stored width percentage, table position, and container width.
40. Mouse movement converts horizontal pixels to a percentage delta and applies a live DOM preview.
41. On mouseup, the preview is restored from stored node attributes, then one setNodeMarkup transaction persists the final width and adjusted offset.
42. closeHistory makes the completed drag one undoable history action.
43. Blur, missing DOM, read-only changes, or plugin destruction cancel safely and restore stored state.

8.3 Row resizing

44. Hit detection activates near the bottom edge of a td or th, or on the explicit row-resize widget.
45. A temporary style element targets the current table wrapper and row index for a live preview.
46. The committed height is clamped to at least 20 pixels.

47. TableMap.positionAt finds every unique cell participating in the row; a visited set prevents duplicate updates for spanning cells.
48. rowHeight is written to each unique cell in one closeHistory transaction.
49. Preview style elements and data markers are removed on commit, cancel, blur, read-only transition, or destruction.

8.4 Dimension normalization rules

Value	Rule
Outer width	Finite numeric value, rounded to one decimal place, clamped to 10-100 percent; invalid values become 100.
Horizontal offset	Finite numeric value, rounded to one decimal place, clamped to 0 through (100 - width); invalid values become 0.
Row height	Finite positive value, rounded to an integer, minimum 20 pixels; invalid values become null or the minimum when clamped.
Container width	Must be finite and greater than zero; otherwise plugins use 1 pixel to avoid division by zero.

9. Borders and Rendered Styling

9.1 Border data model

Borders are table-level boolean attributes rather than transient wrapper classes. This makes them durable across JSON serialization, HTML output, editor remounts, undo/redo, and table node-view updates. Missing values normalize to true for backward compatibility.

9.2 Rendering strategy

KnowledgeBaseTableView applies all border flags as data-table-border-* attributes on the rendered HTML table. tiptap-table.css uses those attributes to make individual edge colors transparent, hide all inner borders, and selectively restore enabled outer edges when inner borders are disabled.

9.3 Toolbar behavior

- All borders toggles all five flags.
- Outer border toggles top, right, bottom, and left together without changing inner borders.
- Inner border controls only the internal grid.
- Individual edge controls update one attribute.
- The Borders toolbar group is active when any border is enabled.

CSS cleanup: Repeated .tiptap and .ProseMirror selectors were consolidated with :is(...). The persistent data-attribute contract remains unchanged.

10. Table Dragging, Positioning, and Reordering

10.1 Drag handle

TableDragHandlePlugin creates a button decoration only for the active table. The handle is hidden and non-draggable in read-only mode. Clicking it creates a table NodeSelection. Starting a drag serializes the table node to clipboard-compatible HTML and text without detaching the decoration.

10.2 Dominant-axis behavior

After a small threshold, the drag locks to horizontal or vertical movement. Horizontal movement adjusts tableOffsetPct. Vertical movement moves the table as a top-level ProseMirror node. Once chosen, the axis does not change during the drag.

Axis	Preview	Commit
Horizontal	Applies a live table margin-left/offset preview and repositions the handle.	Persists tableOffsetPct with setNodeMarkup as one history action.
Vertical	Uses the current target block and pointer position relative to its midpoint.	Deletes and reinserts the entire table node at a valid drop point, then selects it.

10.3 Safe vertical placement

Vertical placement only moves top-level table nodes. createTableMoveTransaction rejects invalid positions, nested table positions, drops inside the dragged table, and unsupported drop points. The drop direction is determined by whether the pointer is above or below the target block's visual midpoint, with movement direction used only as a fallback.

10.4 Row reordering API

reorderTableRow provides a safe command-level API for moving rows inside a table. It is currently tested and available to future UI, but no row-reorder toolbar or drag handle is exposed. The function validates row indexes, uses ProseMirror's moveTableRow, then normalizes the full table so header intent remains anchored to the first row and first column.

11. DOM and Interaction Utilities

11.1 Table DOM helpers

tableDom.ts centralizes all conversions among ProseMirror table positions, node selections, table wrapper DOM nodes, HTMLTableElement instances, event targets, and editor-relative overlay positions. Each helper validates inputs and catches view/DOM lookup failures.

Helper	Responsibility

Helper	Responsibility
getTableNodeAt	Validate a numeric document position and return only a table node.
getActiveTable / getActiveTablePos	Resolve the table around the current selection or table NodeSelection.
mapTablePos	Map a stored position through a transaction and reject deleted/non-table results.
getTableWrapperAtPos / getTableAtPos	Safely resolve the rendered wrapper and HTML table.
getOwnerWindow	Use the editor document's Window for correct instanceof checks and event listeners.
getClosestHTMLElement	Safely find event-target ancestors in the correct browser realm.
requestViewAnimationFrame	Schedule DOM positioning only while the editor view is alive.
positionOverlayAtRect	Convert viewport rectangles to editor-relative overlay geometry.

11.2 Mouse drag session helper

startMouseDragSession owns window-level mousemove, mouseup, and blur listeners. It guarantees cleanup happens once, distinguishes commit from cancellation, and gives plugins one idempotent cancel method for read-only transitions and destruction.

11.3 Row preview helper

rowHeightPreview.ts creates a uniquely identified temporary style element and marks the table wrapper with a matching data attribute. This avoids mutating each cell's inline style during mousemove and allows the preview to be removed deterministically.

12. File-by-File Responsibility Reference

This section maps every table-related implementation and test file to its ownership boundary. Paths are relative to the repository root.

Implementation Files

src/features/editor/extensions/index.tsx

Aspect	Details
Responsibility	Assembles the complete Tiptap extension list.
Key behavior	Disables default TableKit table/cell/header nodes and installs the custom tableExtensions.
Important dependencies	@tiptap/starter-kit, @tiptap/extension-table, table/extensions
Maintenance notes	Any new table node replacement must be coordinated here to avoid duplicate extension names.

src/features/editor/table/extensions/index.ts

Aspect	Details
Responsibility	Composition root for the table subsystem.
Key behavior	Configures the custom table, row-height cells/headers, and three custom ProseMirror plugins.
Important dependencies	KnowledgeBaseTable, TableCellExtensions, RowResizePlugin, TableOuterResizePlugin, TableDragHandlePlugin
Maintenance notes	Plugin extensions are intentionally small wrappers; keep table-wide configuration here.

src/features/editor/table/extensions/TableExtension.ts

Aspect	Details
Responsibility	Owns the custom table node, attributes, node view, and destructive-key behavior.
Key behavior	Persists width/offset/borders; reapplies attributes to DOM; clears selected cell content on Delete/Backspace.
Important dependencies	@tiptap/extension-table Table/TableView, tableDimensions, tableBorders
Maintenance notes	This is the schema contract. Attribute changes require serialization, CSS, and compatibility tests.

src/features/editor/table/extensions/TableCellExtensions.ts

Aspect	Details
Responsibility	Adds rowHeight to tableCell and tableHeader.

Aspect	Details
Key behavior	Normalizes and renders row height consistently for normal and header cells.
Important dependencies	@tiptap/extension-table TableCell/TableHeader, tableDimensions
Maintenance notes	Keep both extensions aligned; rows may contain either node type.

src/features/editor/table/commands/tableCommands.ts

Aspect	Details
Responsibility	Central safe command boundary and header normalization layer.
Key behavior	Validates editor/table state, computes capabilities, preserves headers, updates borders, inserts tables, and reorders rows.
Important dependencies	Tiptap Editor, ProseMirror TableMap/selectedRect/moveTableRow, tableDom
Maintenance notes	Toolbar and future UI should call these wrappers rather than raw table commands.

src/features/editor/table/dom/tableDom.ts

Aspect	Details
Responsibility	Shared ProseMirror-to-DOM lookup and overlay positioning utilities.
Key behavior	Finds active tables, validates positions, maps positions through transactions, resolves DOM safely.
Important dependencies	ProseMirror model/state/view
Maintenance notes	Use these helpers in plugins to avoid duplicated fragile position walking and cross-window instanceof checks.

src/features/editor/table/plugins/RowResizePlugin.ts

Aspect	Details
Responsibility	Interactive row resizing.
Key behavior	Detects row edges, displays a live preview, commits rowHeight to unique row cells, and handles cancellation/read-only state.
Important dependencies	TableMap, tableDom, rowHeightPreview, tableDimensions, mouseDragSession
Maintenance notes	Merged/spanning cells require the visited-cell logic; do not replace it with simple DOM row iteration for persistence.

src/features/editor/table/plugins/TableOuterResizePlugin.ts

Aspect	Details
Responsibility	Interactive outer table width resizing.
Key behavior	Detects the right edge, previews percentage width, adjusts offset, and commits one undoable node attribute change.
Important dependencies	tableDom, tableDimensions, mouseDragSession, ProseMirror decorations/history
Maintenance notes	The right-edge vertical-bound check prevents resize activation beside but outside the table.

src/features/editor/table/plugins/TableDragHandlePlugin.ts

Aspect	Details
Responsibility	Whole-table dragging and horizontal positioning.
Key behavior	Creates the drag handle, serializes drag payload, locks axis, moves top-level table nodes, or persists horizontal offset.
Important dependencies	dropPoint, NodeSelection, tableDom, tableDimensions
Maintenance notes	Vertical drop placement is based on target midpoint; nested/non-table positions fail safely.

src/features/editor/table/resizing/tableDimensions.ts

Aspect	Details
Responsibility	Pure dimension parsing, normalization, clamping, reading, and DOM application.
Key behavior	Enforces width, offset, and row-height invariants and protects against NaN/infinite values.
Important dependencies	Browser HTMLTableElement only
Maintenance notes	Keep geometry math pure and covered by unit tests; plugins should delegate to this module.

src/features/editor/table/resizing/rowHeightPreview.ts

Aspect	Details
Responsibility	Temporary live row-height styling.
Key behavior	Creates, updates, and removes unique preview style elements scoped to a table wrapper.
Important dependencies	tableDimensions
Maintenance notes	Preview state must always be restored on cancel, blur, read-only transition, and destruction.

src/features/editor/table/utils/tableBorders.ts

Aspect	Details
Responsibility	Border attribute types, defaults, normalization, and DOM application.
Key behavior	Defaults missing borders to enabled and writes data-table-border-* attributes.
Important dependencies	HTMLTableElement
Maintenance notes	This module is the compatibility and type-safety boundary for border flags.

src/features/editor/table/utils/mouseDragSession.ts

Aspect	Details
Responsibility	Reusable mouse interaction lifecycle.
Key behavior	Registers movement/commit/cancel listeners and guarantees one-time cleanup.
Important dependencies	Window mouse and blur events
Maintenance notes	Keep this independent of specific table behaviors so all resize plugins share the same lifecycle semantics.

src/features/editor/table/toolbar/TableControls.tsx

Aspect	Details
Responsibility	Contextual table toolbar and typed toolbar-state selector.
Key behavior	Shows valid commands, active headers, and border flags; routes every action through safe command wrappers.
Important dependencies	useEditorState, ToolbarPrimitives, tableCommands, tableBorders
Maintenance notes	Do not call raw editor table commands here; capability and action paths must stay consistent.

src/features/editor/table/toolbar/TableCreationPicker.tsx

Aspect	Details
Responsibility	Accessible table-size picker.
Key behavior	Provides grid and manual size selection in a Floating UI dialog and clamps manual values.
Important dependencies	@floating-ui/react, React state
Maintenance notes	The final insertion still goes through insertTable; UI validation is not the sole safety boundary.

src/features/editor/table/toolbar/TableIcons.tsx

Aspect	Details
Responsibility	Table-specific inline SVG icon components.
Key behavior	Provides consistent icon size and accessible decorative SVG markup.
Important dependencies	ToolbarPrimitives ICON_SIZE
Maintenance notes	Keep icons presentation-only; behavior belongs in controls and commands.

src/features/editor/table/toolbar/index.ts

Aspect	Details
Responsibility	Public toolbar exports.
Key behavior	Exports TableControls and TableCreationPicker.
Important dependencies	Table toolbar components
Maintenance notes	Keep the public surface small.

src/features/editor/components/toolbar/EditorToolbar.tsx

Aspect	Details
Responsibility	Main editor toolbar integration point.
Key behavior	Renders TableCreationPicker and TableControls; routes creation through insertTable.
Important dependencies	Table toolbar exports and tableCommands
Maintenance notes	The contextual table toolbar is rendered below the global formatting toolbar.

src/features/editor/styles/tiptap-table.css

Aspect	Details
Responsibility	All visual table, selection, border, and interaction-handle styling.
Key behavior	Uses persistent data attributes and :is(.tiptap, .ProseMirror) selectors.
Important dependencies	Custom table attributes and plugin-generated classes
Maintenance notes	CSS selectors are part of the rendering contract; update alongside attribute/class changes.

src/app/globals.css

Aspect	Details
Responsibility	Global stylesheet entry point.
Key behavior	Imports tiptap-table.css so table rendering applies throughout the editor route.
Important dependencies	Next.js App Router global CSS
Maintenance notes	The local Next.js 16 guidance permits global CSS from the app tree; keep import ordering predictable.

Test Files

Test file	Coverage responsibility
src/features/editor/table/commands/tableCommands.test.ts	Unit/integration coverage for safe commands, invalid states, header preservation, selected rectangles, borders, undo/redo, and row reordering.
src/features/editor/table/extensions/tableExtensions.integration.test.ts	Editor-level coverage for serialization, drag handle behavior, merge/split, selected-cell deletion, read-only mode, and move transactions.
src/features/editor/table/toolbar/TableControls.test.ts	Toolbar-state coverage for inactive editors, full-table selection capability rules, and persisted border synchronization.
src/features/editor/table/resizing/tableDimensions.test.ts	Pure dimension normalization plus row preview and outer-edge hit-detection coverage.
src/features/editor/table/utils/tableBorders.test.ts	Border defaults and rendered data-attribute coverage.
src/features/editor/table/utils/mouseDragSession.test.ts	Movement forwarding, one-time commit, blur cancellation, and idempotent explicit cancellation.
tests/integration/table-editor.spec.ts	Browser-level behavior for outer/internal resizing, row resizing, borders, table block movement, horizontal positioning, focus, and portal toolbar behavior.

13. Edge-Case Behavior Matrix

Edge case	Expected behavior	Mechanism
No active editor	Creation/action helpers return false; contextual controls are inactive.	Safe wrapper validation
Destroyed editor	Commands and toolbar state return false/inactive without touching view/state.	editor.isDestroyed checks
Read-only editor	Mutation controls hidden/inactive; resize and drag sessions cancel; handles become inert.	isEditable/view.editable checks
No selected table	Table actions return false and TableControls is hidden.	getActiveTable + isActive('table')
Invalid/malformed table map	TableMap access is caught and action returns false.	getTableMap / plugin map helpers

Edge case	Expected behavior	Mechanism
Insert row above header	New first row is header; former header row is demoted.	Edge header normalization
Delete header row	Next row is promoted to header.	Snapshot header intent + normalization
Delete header column	New first column is promoted.	Snapshot header intent + normalization
Reorder first row	Header semantics remain on first row/column.	Full-table normalization
Select every row/column	Delete row/column toolbar actions are disabled; commands return false.	selectedRect guard
Delete all selected cells	Cell content is cleared; table remains.	deleteCellSelection shortcut override
Merge/split invalid selection	Capability is false and action returns false.	canRunTableCommand / action wrapper
Resize with invalid numeric values	Values fall back to safe width/offset/minimum height.	tableDimensions normalization
Pointer beside outer edge but outside table height	Outer resize does not activate.	isNearTableRightEdge vertical bounds
Table DOM disappears during drag	Session cancels and stored state is restored.	DOM lookup + cancel path
Window loses focus during mouse drag	Resize cancels and listeners are removed.	mouseDragSession blur handling
Table moved/deleted during plugin state	Stored table position maps or becomes null.	mapTablePos
Drop inside dragged table	Move transaction returns null.	createTableMoveTransaction validation
Horizontal offset exceeds space	Offset clamps to 100 - width.	clampTableOffsetPct

14. Hardening Improvements

14.1 Command and toolbar safety

- Introduced one typed safe-command module for all table UI mutations.
- Added null, destroyed-editor, read-only, no-table, invalid-map, and invalid-input guards.
- Wrapped underlying commands in false-returning exception boundaries.
- Made toolbar capability checks selection-aware for full-table row/column selections.
- Routed table insertion and border updates through the same safety boundary.

14.2 Header correctness

- Preserved header intent across insertion, deletion, undo, redo, and row reordering.
- Demoted displaced former header rows/columns instead of allowing duplicate header edges.

- Added full-table normalization for row reordering.
- Added focused tests for first-row/header-row and first-column/header-column behavior.

14.3 Selection and destructive behavior

- Overrode Tiptap's full-cell-selection table deletion shortcut.
- Delete and Backspace now clear selected cell content without deleting the grid.
- Kept explicit whole-table deletion as a deliberate toolbar action.

14.4 Resize and drag reliability

- Protected width/offset clamping from NaN and infinite values.
- Restricted outer resize hit detection to the actual vertical table bounds.
- Added guarded commit transactions for row height, outer width, and horizontal offset.
- Validated top-level table positions before moving a table node.
- Changed vertical drop placement to use target-block midpoint geometry.

14.5 Readability and modularity

- Kept responsibilities in dedicated extensions, commands, plugins, resizing helpers, DOM helpers, toolbar components, and utilities.
- Consolidated repeated table CSS selectors with :is(...).
- Added explicit toolbar-state typing and a complete inactive state.
- Retained the existing table folder structure because it already reflects clean ownership boundaries.

15. Test and Verification Strategy

15.1 Verification completed

Check	Result	What it verifies
npm test	40 passed	Commands, extensions, toolbar state, utilities, resizing math, selection behavior, serialization, and plugin helpers.
npm run test:integration	10 passed	Real browser behavior for resize, borders, row height, table movement, offsets, toolbar focus, and portal UI.
npm run lint	Passed	ESLint and Next.js lint rules.
npx tsc --noEmit	Passed	Type correctness across the project.
npm run build	Passed	Next.js 16.2.7 production compilation, TypeScript stage, and static generation.
In-app browser inspection	Passed	New table rendered with header row, drag handle, toolbar, width/offset, and border attributes.

15.2 Test layering

Pure math and attribute behavior is tested at the utility layer. Command behavior is tested with a real Tiptap Editor in jsdom. Plugin integration and serialization are tested with the editor and rendered DOM. Geometry-sensitive behavior is finally verified in Playwright against the running Next.js application.

Testing principle: A command test proves document semantics. A browser test proves pointer geometry and rendered behavior. Both are required for resize and drag features.

16. Remaining Risks and Production Concerns

Concern	Current status	Recommended follow-up
Row reorder UI	Safe reorderTableRow API exists, but no user-facing row reorder control is exposed.	Add UI only when product behavior and accessibility are defined.
Merged-cell row resize	TableMap and visited-cell logic protects persistence, but complex rowspan layouts remain browser-layout dependent.	Add browser cases with rowspans and colspans before expanding merged-cell workflows.
Input modalities	Custom resize and drag interactions are mouse-oriented.	Add Pointer Events/touch support and keyboard-accessible alternatives if mobile/tablet use is required.
Browser coverage	Automated browser tests currently target Chrome.	Add Firefox/WebKit runs in CI if cross-browser support is a production requirement.
Very large tables	Manual insertion allows up to 100 rows by 20 columns.	Measure editor/update performance and consider stricter product limits or virtualization strategies.
Imported malformed HTML	Commands fail safely, but malformed tables may remain non-editable until fixed.	Consider an explicit repair/import validation flow using fixTables and user feedback.
Border semantics with spanning cells	CSS edge selectors use first/last child visual positions.	Add dedicated browser tests for complex rowspan/colspan border combinations.
Collaboration/concurrent editing	Position mapping is defensive inside local transactions.	Re-evaluate plugin session cancellation and mapped positions before enabling collaborative editing.
Read-only initialization	Tiptap column resizing plugins are configured based on initial editor editability.	Verify toggling editable mode repeatedly if read-only/edit transitions become frequent.

Production posture: The table subsystem now fails safely and has strong local/browser coverage. The primary remaining risks are advanced layout combinations, non-mouse accessibility, cross-browser geometry, and performance at product-limit table sizes.

17. Safe Extension and Maintenance Guide

17.1 Adding a new table attribute

50. Define the attribute and backward-compatible default in the appropriate custom extension or utility.
51. Normalize unknown/imported values before using them.
52. Render a stable HTML/data-attribute contract if CSS or export depends on it.
53. Apply the attribute in KnowledgeBaseTableView when the editable DOM needs explicit synchronization.
54. Route toolbar updates through a safe command helper.
55. Add JSON, HTML, restored-editor, and browser-rendering tests.

17.2 Adding a new structural command

56. Add a typed command name or dedicated wrapper in tableCommands.ts.
57. Validate editor, editability, active table, selection, and command-specific inputs.
58. Determine whether the command can disturb edge-header semantics.
59. Keep all related document mutations inside one transaction/history action.
60. Expose capability and action paths through TableControls without calling raw commands.
61. Test invalid state, first-row/header behavior, selected-cell behavior, and undo/redo.

17.3 Adding a geometry-heavy interaction

62. Keep persistent values on nodes and live previews in DOM/plugin session state.
63. Use tableDom helpers for position and DOM resolution.
64. Use startMouseDragSession or a pointer-equivalent lifecycle with guaranteed cleanup.
65. Map stored positions through transactions and cancel if the target disappears.
66. Restore preview state before committing the final transaction.
67. Use closeHistory so the interaction becomes one undoable action.
68. Test cancellation, blur, read-only transition, invalid DOM, undo, and browser geometry.

17.4 Review checklist

- No table UI calls raw mutation commands when a safe wrapper exists.
- Commands return false for invalid state and do not throw.
- Header-row and header-column intent survives structural changes.
- Full-table selections do not expose misleading row/column deletion controls.
- Selected-cell Delete/Backspace preserves the table.
- Preview DOM state is always cleaned up.
- Persistent attributes serialize through JSON and HTML.
- Toolbar state updates with selection and attribute changes.

- Unit, editor integration, browser integration, TypeScript, lint, and build checks pass.

Appendix A. Command and Attribute Quick Reference

User intent	Preferred API	Persistent mutation
Create table	insertTable(editor, rows, cols)	Inserts a table with withHeaderRow: true.
Insert/delete row or column	runTableStructureCommand	Structural change plus header normalization.
Merge/split cells	runTableActionCommand	Tiptap merge/split transaction.
Delete table	runTableActionCommand(editor, 'deleteTable')	Explicit table deletion.
Toggle header row/column	runTableActionCommand	Changes cell node types through Tiptap.
Update borders	updateTableBorders	Updates table border attributes.
Move row	reorderTableRow	moveTableRow plus full header normalization.
Resize outer table	TableOuterResizePlugin	Updates tableWidthPct and possibly tableOffsetPct.
Resize row	RowResizePlugin	Updates rowHeight on unique participating cells.
Move/position whole table	TableDragHandlePlugin	Moves table node vertically or updates tableOffsetPct horizontally.

Appendix B. Key Invariants

- The first row and first column are the only semantic header edges.
- Table width is between 10 and 100 percent.
- Horizontal offset is between 0 and 100 minus table width.
- Row height is at least 20 pixels.
- Missing border attributes mean enabled.
- A resize or drag commit is one history action.
- A stale or invalid table/editor state returns false or null instead of throwing.
- Selected-cell keyboard deletion clears content rather than deleting the table.
- Toolbar state must describe the command that will actually run.

End state: The subsystem is organized around explicit ownership, safe command boundaries, durable node attributes, deterministic cleanup, and layered verification. Future table changes should preserve those properties.